

Homework Progetto#3

Relazione

Giorgia Maria Musumeci - 1000061185

INTRODUZIONE

Con questo terzo elaborato, il progetto di monitoraggio del traffico aereo raggiunge la sua fase conclusiva, completando l'evoluzione da semplice applicazione di raccolta dati a sistema distribuito Cloud Native.

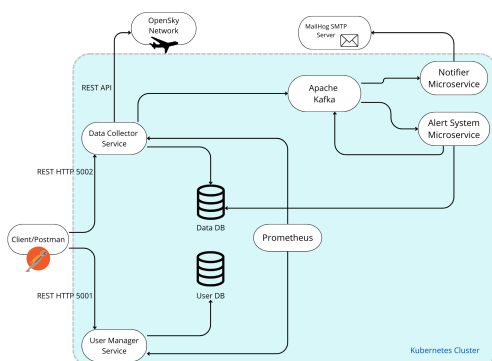
Se nelle fasi precedenti l'attenzione era focalizzata sulle logiche applicative, quindi dalla consultazione dei dati di volo (HW1) all'implementazione di un'architettura event-driver per la gestione degli alert tramite Kafka (HW2), in questa fase il focus si sposta decisamente sull'infrastruttura e sulla Observability.

La modifica strutturale più significativa riguarda la strategia di deployment: l'orchestrazione locale basata su Docker Compose è stata abbandonata in favore di una gestione tramite cluster Kubernetes (implementato localmente con Kind).

Parallelamente, per garantire il controllo sulle performance del sistema, è stato introdotto un meccanismo di White-box-monitoring.

L'architettura è stata arricchita con un server Prometheus e i microservizi principali sono stati strumentati a livello di codice per esporre metriche in tempo reale (come il conteggio delle richieste e la latenza delle operazioni).

DIAGRAMMA ARCHITETTURALE



L'architettura del sistema, inizialmente basata su container Docker orchestrati localmente, è stata evoluta in questa fase verso un modello Cloud Native su Kubernetes. Il core logico mantiene l'approccio Event-Driven, dove un Message Broker garantisce il disaccoppiamento tra la raccolta dati e la logica di notifica/alerting.

A livello infrastrutturale, è stato introdotto il layer di Observability tramite Prometheus.

Come mostrato in figura, il sistema si compone ora dei seguenti elementi, tutti eseguiti come Pod all'interno del cluster:

1. **User Manager Service**: microservizio responsabile della gestione degli utenti (registrazione e cancellazione). Espone un'interfaccia REST verso l'esterno e un server gRPC verso l'interno per consentire agli altri servizi di verificare l'esistenza degli utenti. È stato strumentato per esporre metriche sulle richieste HTTP.
2. **Data Collector Service**: microservizio "core" che gestisce la logica di business. Si occupa di raccogliere le preferenze degli utenti, verificare la loro esistenza tramite gRPC, scaricare ciclicamente i dati da OpenSky Network tramite uno scheduler interno, e agire come Kafka Producer pubblicando i dati di volo sul topic `flight_data`.
3. **User DB**: database relazionale (MySQL) dedicato esclusivamente ai dati anagrafici degli utenti.
4. **Data DB**: database relazionale (MySQL) dedicato alla memorizzazione del interessi e dello storico dei voli scaricati.
5. **Apache Kafka**: Message Broker centrale che gestisce i flussi di dati asincroni tra i microservizi. Gestisce due topic principali: `flight_data` per i dati grezzi sui voli inviati dal collector e `alerts` per le notifiche di allarme generate dall'Alert System.
6. **Alert System**: nuovo microservizio che agisce come Kafka Consumer. Ascolta il topic `flight_data`, consulta il database per

recuperare le soglie di allerta degli utente e, se trova una violazione produce un messaggio di allerta sul topic alerts.

7. **Notifier System**: microservizio finale della catena. Consuma i messaggi dal topic alerts e si occupa di formattare e inviare le email di notifica.
8. **MailHog**: server SMTP fittizio utilizzato in ambiente di sviluppo per intercettare e visualizzare le email inviate dal Notifier System, simulando la casella di posta dell'utente.
9. **Prometheus**: il nuovo componente infrastrutturale che esegue lo scraping (lettura) delle metriche esposte dai servizi User Manager e Data Collector sulla porta dedicata.

SCELTE PROGETTUALI

Per lo sviluppo e il deployment del sistema sono state adottate le seguenti scelte tecniche.

È stato mantenuto l'uso di Python con framework Flask. Questa scelta garantisce continuità con le fasi precedenti e facilita l'integrazione della libreria prometheus-client, necessaria per il monitoraggio White-box.

Nella registrazione utenti continua ad essere implementata la logica At-Most-Once utilizzando la logica basata su request_id per garantire l'idempotenza delle richieste anche in ambiente distribuito.

Il Data Collector implementa il pattern Circuit Breaker (libreria pybreaker) per proteggere il sistema da fallimenti ripetuti delle API esterne di OpenSky Network.

Il passaggio a **Kubernetes** ha richiesto l'uso della Downward API. Poiché i container sono effimeri e isolati, non conoscono nitidamente su quale macchina fisica (nodo) stanno girando. Per soddisfare il requisito di monitoraggio, è stata iniettata la variabile d'ambiente **NODE_NAME** nelle specifiche del Deployment. Questo permette al codice Python di leggere il nome del nodo e allegarlo come "label" alle metriche inviate a Prometheus, rendendo possibile filtrare le performance per nodo specifico.

```
- name: NODE_NAME
  valueFrom:
    fieldRef:
      fieldPath: spec.nodeName
```

Durante il deployment su **Kind**, è emersa una criticità legata alle variabili d'ambiente automatiche di Kubernetes, risolta configurando esplicitamente i listener di Kafka.

MONITORAGGIO E METRICHE

Per garantire l'osservabilità del sistema distribuito, è stato adottato un approccio di White-box Monitoring. A differenza del monitoraggio black-box (che si limita a verificare se un servizio risponde), questa metodologia prevede che sia l'applicazione stessa a esporre dati sul suo stato interno.

Il sistema si basa su **Prometheus**, all'interno del cluster Kubernetes. Prometheus opera in modalità PULL:

1. Il server legge la configurazione (ConfigMap) dove sono definiti i target (servizi da monitorare).
2. Ogni 5 secondi, contatta gli endpoint HTTP / metrics dei microservizi User Manager e Data Collector.
3. Salva le serie temporali nel suo database interno (TSDB).

Le metriche sono state progettate per coprire sia il traffico che le prestazioni:

User Manager Service

- Counter : user_manager_requests_total
conta il numero totale di richieste ricevute dal servizio permettendo di calcolare il tasso di richieste al secondo e monitorare il carico.
- Gauge :
user_manager_op_duration_seconds
misura il tempo di latenza dell'ultima operazione gestita.

Data Collector Service

- Counter :
data_collector_operations_total
monitora la stabilità del job di scaricamento e l'intervento del Circuit Breaker.
- Gauge :
data_collector_job_duration_seconds
misura quanto tempo impiega il sistema a scaricare e processare i dati da OpenSky e se questo valore cresce troppo, indica problemi di rete o sovraccarico del Data DB.

DETTAGLIO DEL DEPLOYMENT SU KUBERNETES

L'infrastruttura è stata descritta in modo dichiarativo tramite file YAML, suddivisi per ambito logico per facilitare la manutenzione.

1. Infrastructure Layer (infrastructure.yaml)

Come database abbiamo due deployment distinti per MySQL, esposti internamente tramite Service ClusterIP sulla porta 3306. Message Broker (Kafka): è stata utilizzata l'immagine confluentinc/cp-kafka in modalità Kraft per ridurre il consumo di risorse nel cluster locale Kind.

È stato necessario rinominare il servizio in my-kafka e configurare esplicitamente KAFKA_ADVERTISED_LISTENERS per evitare conflitti con le variabili d'ambiente automatiche di Kubernetes (KAFKA_PORT) che causavano il crash del container.

2. Application Layer (apps.yaml)

User Manager e Data Collector sono configurati come Deployment. Espongono le porte 5001/5002 (HTTP) e 50051 (gRPC) tramite Service ClusterIP. Le variabili d'ambiente configurano gli indirizzi dei DB e di Kafka utilizzando i nomi DNS interni del cluster. Alert e Notifier System sono configurati come Deployment semplici senza Service associato, in quanto agiscono come "Consumer" puri (iniziano loro la connessione verso Kafka) e non ricevono traffico in ingresso.

3. Monitoring Layer (prometheus.yaml)

ConfigMap contiene il file prometheus.yml con le regole di scraping statico verso i nomi DNS dei servizi (user-manager e data-collector). Il servizio Prometheus è esposto tramite NodePort (o accessibile via Port-Forwarding sulla porta 9090) per consentire la visualizzazione delle dashboard dall'esterno del cluster.

4. Alert System Layer (alert-system.yaml)

Il file è definito come un Deployment che agisce come Consumer puro di Kafka. Una peculiarità di questo componente è l'assenza di un oggetto Service associato: non dovendo ricevere traffico in ingresso, è il pod stesso ad avviare le connessioni verso il Broker e verso il Data DB sfruttando le variabili d'ambiente.

5. Notifier System Layer (notifier.yaml)

Questo layer gestisce la parte di notifica e include due componenti:

- Notifier System: configurato come Deployment, consuma gli eventi dal topic alerts e li converte in email. Come l'Alert System, opera senza Service in ingresso, connettendosi in uscita al server SMTP.
- MailHog: è il server SMTP fittizio per il testing. A differenza del Notifier, dispone di un Service che espone la porta 1025 per la ricezione delle email e la porta 8025 accessibile via

Port-Forwarding, permettendo la verifica delle notifiche inviate.

PATTERNS DI COMUNICAZIONE

L'applicazione è una piattaforma distribuita per il monitoraggio aereo. La comunicazione tra i microservizi segue un approccio ibrido per massimizzare l'efficienza:

- **Sincrona (gRPC)**: utilizzata tra Data Collector e User Manager per la verifica dell'esistenza utente.
- **Asincrona (Kafka)**: utilizzata per il flusso dati massivo (Voli) e critico (Allarmi). Questo disaccoppia il produttore del dato (Collector) dal consumatore (Alert System), garantendo che un rallentamento nell'invio delle email non blocchi lo scaricamento dei voli.

I pattern implementati sono:

- **At-Most-Once (idempotenza)**: implementata nello *User Manager* tramite `request_id`. Garantisce che, in caso di retry dovuti alla rete, un utente non venga registrato due volte.
- **Circuit Breaker**: implementato nel *Data Collector* con libreria `pybreaker`. Protegge il sistema dai fallimenti dell'API esterna OpenSky, interrompendo le richieste temporaneamente dopo una serie di errori consecutivi.

LISTA API IMPLEMENTATE

Il sistema espone le seguenti interfacce RESTful verso i client esterni (es. Postman):

Microservizio User Manager (Porta 5001)

- POST /register : registra un nuovo utente
- GET /metrics : espone le metriche per Prometheus.

Microservizio Data Collector (Porta 5002)

- POST /add_interest : aggiunge un aeroporto da monitorare.
- GET /stats/last//<airport_code>: restituisce l'ultimo set di dati scaricato per l'aeroporto.
- GET /stats/average/<airport_code>/>days> : calcola la media dei voli negli ultimi X giorni.
- GET /metrics: espone le metriche per Prometheus.

TEST E VALIDAZIONE DEL SISTEMA

La validazione del sistema migrato su Kubernetes è stata effettuata attraverso una serie di test funzionali e infrastrutturali, verificando sia la correttezza della logica di business sia l'effettiva raccolta delle metriche. Gli strumenti utilizzati sono stati: Postman (per le chiamate API), `kubectl` (per l'introspezione del cluster) e la Dashboard di Prometheus.

Di seguito tre scenari di test principali:

Verifica del Deployment

Verificare che l'orchestrazione Kubernetes mantenga i servizi attivi. È stato eseguito il comando `kubectl get pods` verificando che tutti i container (inclusi Kafka e i DB) fossero in stato Running.

```
(.venv) giorgiamusumeci@Host-007 Homework_3 % kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
data-collector-856f78797d-9cpmc     1/1     Running   0           31s
data-db-6dbf4b7c8f-cc6fc           1/1     Running   0          6d23h
kafka-5c74b798b7-7sl5c             1/1     Running   0          3d22h
prometheus-6c5c964b56-znq6q        1/1     Running   0          6d23h
user-db-6fbdf8ff69-fv9wk           1/1     Running   0          6d23h
user-manager-66dbf5d87d-md9hg       1/1     Running   0          6d23h
```

È stata simulata la caduta del Data Collector cancellando manualmente il pod.

```
(.venv) giorgiamusumeci@Host-007 Homework_3 % kubectl delete pod data-collector-856f78797d-tq7db
pod "data-collector-856f78797d-tq7db" deleted from default namespace
```

Kubernetes ha rilevato l'assenza del pod e ne ha avviato automaticamente una nuova istanza, garantendo la continuità del servizio.

```
(.venv) giorgiamusumeci@Host-007 Homework_3 % kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
data-collector-856f78797d-tq7db     1/1     Running   0          3d22h
data-db-6dbf4b7c8f-cc6fc           1/1     Running   0          6d23h
kafka-5c74b798b7-7sl5c             1/1     Running   0          3d22h
prometheus-6c5c964b56-znq6q        1/1     Running   0          6d23h
user-db-6fbdf8ff69-fv9wk           1/1     Running   0          6d23h
user-manager-66dbf5d87d-md9hg       1/1     Running   0          6d23h
```

Verifica del Flusso Event-Driven (Kafka)

Confermare che la comunicazione asincrona funzioni all'interno della rete del cluster.

Esecuzione:

1. Il Data Collector ha scaricato i voli e prodotto un messaggio `flight_data`.

2. È stato verificato dai log dell'Alert System (`kubectl logs -l app=alert-system`) l'avvenuto consumo del messaggio.

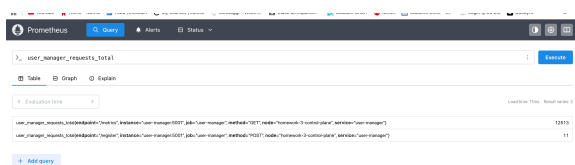
I messaggi sono stati correttamente instradati tramite il servizio `my-kafka`, confermando la corretta configurazione dei listener Kraft.

Verifica del Monitoraggio (Prometheus)

Verificare l'esposizione e la raccolta delle metriche custom.

Esecuzione:

1. Sono state inviate 10 richieste di registrazione allo User Manager.
2. È stata interrogata la dashboard di Prometheus tramite la query `user_manager_request_total`.



Il grafico ha mostrato un incremento del contatore corrispondente al numero di richieste effettuate. Inoltre, l'etichetta `node="homework-3-control-plane"` conferma che la Downward API sta iniettando correttamente le informazioni del nodo all'interno del Po', permettendo a Prometheus di tracciare la provenienza delle metriche.

CONCLUSIONE

L'elaborato ha permesso di trasformare un'applicazione inizialmente monolitica o gestita tramite script locali in un vero o proprio sistema distribuito Cloud Native. La migrazione da Docker Compose a Kubernetes (Kind) ha rappresentato il passaggio chiave verso un'infrastruttura di livello enterprise, introducendo concetti fondamentali come l'auto-healing dei Pod e la gestione dichiarativa delle risorse.

L'integrazione di Apache Kafka ha consolidato l'architettura Event-Driven, garantendo un disaccoppiamento efficace tra la logica di business e quella di notifica, aumentando la resilienza complessiva del sistema. Infine, l'implementazione del White-box Monitoring con Prometheus ha fornito la visibilità necessaria per operare il sistema di produzione, permettendo non solo di sapere se i servizi sono

attivi, ma anche come stanno performando in tempo reale.

In conclusione, il sistema ha dimostrato di soddisfare pienamente i requisiti di scalabilità, resilienza e osservabili tipici delle moderne architetture a microservizi.